LAPORAN UJIAN TENGAH SEMESTER

PEMROGRAMAN SISI SERVER (PSS)

Nama : Faishal Rasyid Rusianto

NIM : A11.2024.15869

Kelas : A11.4703

Program Studi : Teknik Informatika

Universitas : Universitas Dian Nuswantoro

Mata Kuliah : Pemrograman Sisi Server

Dosen Pengampu : Suprayogi, M.Kom.

Periode : Semester 5 - UTS 2026

Laporan ini memuat jawaban soal UTS beserta bukti eksekusi nyata Docker Compose, Django ORM, dan eksperimen optimasi database pada proyek sederhana Learning Management System (LMS).

DAFTAR ISI

- Soal 1 - Backend Fundamentals
- Soal 2 - Arsitektur Sistem
- Soal 3 - Docker Compose
- Soal 4 - Django ORM
- Soal 5 - Database Optimization
- Pengujian Tambahan
- Kesimpulan
- Link Repository GitHub

SOAL 1 - BACKEND FUNDAMENTALS (10 poin)

1.1 Perbedaan Frontend dan Backend

Frontend adalah bagian aplikasi yang berjalan di sisi pengguna (browser) dan bertanggung jawab atas tampilan serta interaksi: HTML untuk struktur, CSS untuk gaya, dan JavaScript untuk perilaku. Frontend tidak boleh menyimpan logika bisnis rahasia karena kode di browser bisa dilihat siapa saja.

Backend adalah bagian yang berjalan di server dan menangani data serta aturan bisnis: menerima request, memproses logika (otorisasi, validasi, perhitungan), berbicara ke database, lalu mengirim response (biasanya JSON). Pada proyek UTS ini backend ditulis dengan Django (Python) dan databasenya PostgreSQL.

1.2 Peran Django, PostgreSQL, dan Docker

- Django : framework backend Python. Menyediakan ORM (pemetaan class model ke tabel database), routing URL, view, sistem migrasi skema, dan admin bawaan. Django menjadi otak aplikasi LMS (users, course, lesson, dan lain-lain).
- PostgreSQL : sistem manajemen basis data relasional (RDBMS). Menyimpan data user, course, enrollment, assignment, submission secara permanen, menjaga integritas lewat primary key, foreign key, unique constraint, dan index.
- Docker : alat containerisasi. Melalui Docker Compose, aplikasi Django (service app) dan PostgreSQL (service db) dijalankan sebagai container terpisah namun saling terhubung dalam satu jaringan. Lingkungan jadi konsisten di mesin mana pun tanpa menginstal Python/PostgreSQL manual.

1.3 Alur Request sampai Response: Menampilkan Daftar Course

Ketika pengguna meminta daftar course, alurnya kurang lebih sebagai berikut:

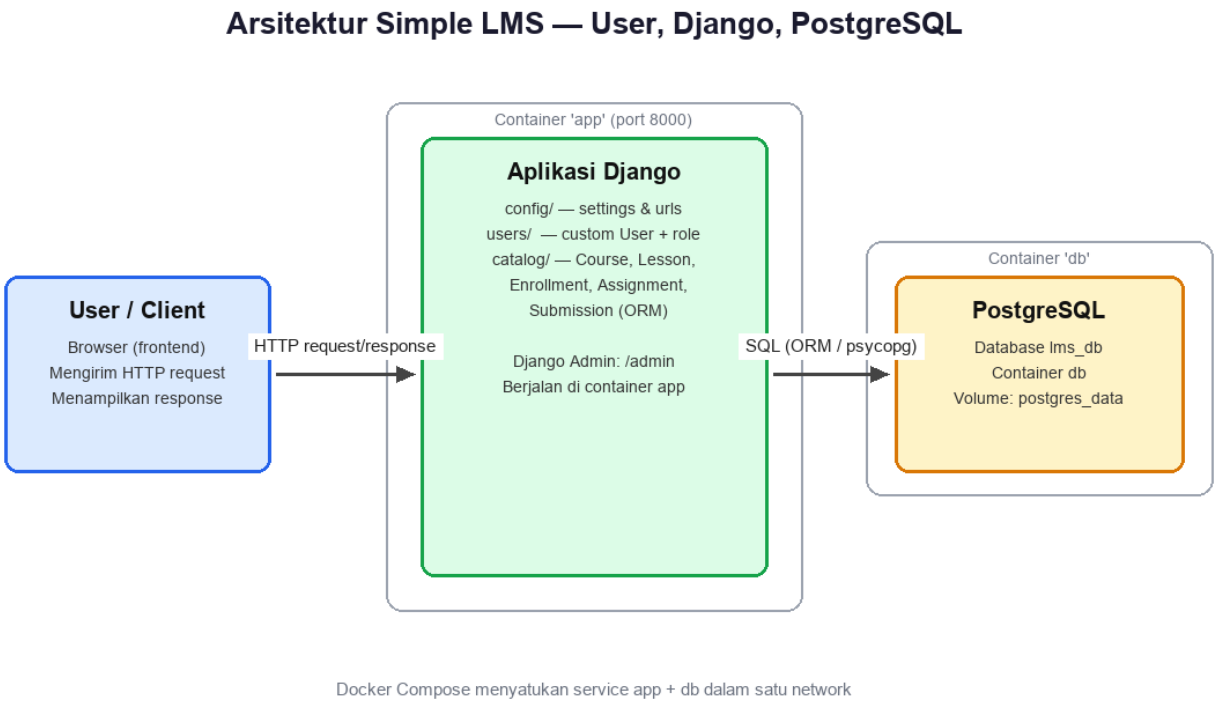
- 1. Browser mengirim HTTP GET ke URL aplikasi, misalnya /api/courses/.
- 2. Docker meneruskan request tersebut ke container app (port 8000) tempat server Django (runserver) berjalan.
- 3. Django mencocokkan URL pada config/urls.py lalu memanggil view yang sesuai, misalnya CourseViewSet atau fungsi course_list di catalog/views.py.
- 4. View meminta data lewat ORM: Course.objects.select_related("lecturer").filter(is_active=True). ORM mengubahnya menjadi perintah SELECT SQL.
- 5. PostgreSQL (container db) mengeksekusi SQL tersebut dan mengembalikan baris-baris course.
- 6. View mengubah objek Python menjadi response JSON (kode, judul, nama dosen).
- 7. Response dikirim balik melewati container app ke browser, lalu frontend menampilkan daftar course.

Singkatnya: browser -> Docker -> Django (URL -> view -> ORM) -> SQL -> PostgreSQL -> data kembali -> JSON -> browser. Frontend hanya mengambil data, sedangkan seluruh aturan dan penyimpanan ditangani backend.

SOAL 2 - ARSITEKTUR SISTEM (15 poin)

2.1 Diagram Arsitektur User -> Django -> PostgreSQL

Berikut diagram aritektur sistem LMS ini, lengkap dengan pembagian dua container pada Docker Compose:



Gambar 2.1 - Arsitektur sistem LMS: User -> Django (container app) -> PostgreSQL (container db).

2.2 Fungsi Tiap Berkas Penting

config/settings.py

Pengaturan inti proyek Django: konfigurasi database PostgreSQL (dibaca dari environment variable DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT lewat os.environ), pendaftaran aplikasi (users, catalog), AUTH_USER_MODEL = users.User (model user kustom), middleware, dan Debug Toolbar untuk inspeksi query.

users/models.py

Mendefinisikan model User kustom yang mewarisi AbstractUser. Ditambahkan field role dengan pilihan ADMIN, LECTURER (dosen), dan STUDENT (mahasiswa) memakai TextChoices, sehingga satu tabel user dapat mewakili dosen sekaligus mahasiswa dan menjadi tujuan foreign key pada model lain.

```
class User(AbstractUser):
    class Role(models.TextChoices):
        ADMIN = "ADMIN", "Administrator"
        LECTURER = "LECTURER", "Dosen"
        STUDENT = "STUDENT", "Mahasiswa"

    role = models.CharField(max_length=20, choices=Role.choices,
                            default=Role.STUDENT)
```

catalog/models.py

Pusat skema database LMS. Berisi model Course (kode unik, judul, dosen via ForeignKey), Lesson (materi milik satu course), Enrollment (mahasiswa terdaftar pada course), Assignment (tugas suatu course), dan Submission (jawaban mahasiswa beserta nilai). Relasi antar model memakai ForeignKey dengan related_name, on_delete, unique constraint, dan index performa.

```
class Course(models.Model):
    code = models.CharField(max_length=20, unique=True)
    title = models.CharField(max_length=200)
    lecturer = models.ForeignKey(settings.AUTH_USER_MODEL,
        on_delete=models.PROTECT, related_name="courses_taught",
        limit_choices_to={"role": "LECTURER"})
    is_active = models.BooleanField(default=True)

    class Meta:
        indexes = [models.Index(fields=["is_active", "title"])]
```

catalog/views.py

Berisi logika permintaan data: menerima request HTTP, memilih QuerySet yang tepat (misalnya Course.objects.select_related("lecturer") agar tidak terjadi masalah N+1), lalu mengubah hasilnya menjadi response JSON. View menjadi penghubung antara URL/endpoint dan model.

2.3 Kenapa Logika Tidak Ditulis dalam Satu Berkas Saja

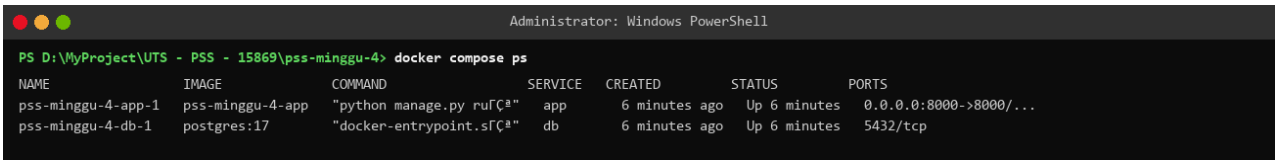
Jika semua logika (pengaturan, model, view, user) ditumpuk dalam satu file, proyek akan sulit dirawat: sulit dilacak saat terjadi error, berkas cepat panjang, dan beberapa bagian tidak bisa dipakai ulang. Dengan dipisah per modul, setiap berkas punya tanggung jawab tunggal (single responsibility).

- Pemisahan tugas : config hanya urusan pengaturan, users khusus identitas, catalog khusus data akademik.
- Kolaborasi tim : beberapa orang dapat mengerjakan aplikasi berbeda tanpa saling mengedit berkas yang sama.
- Skalabilitas : modul baru (misalnya sertifikat atau forum) tinggal ditambah sebagai aplikasi Django baru.
- Perawatan & pengujian : perubahan pada view tidak berisiko merusak model, dan tiap modul dapat diuji terpisah seperti pada proyek ini (perintah manage.py test catalog).

SOAL 3 - DOCKER COMPOSE (15 poin)

3.1 Bukti Container Berjalan (docker compose ps)

Setelah menjalankan docker compose up -d, kedua container aplikasi dan database berstatus Up dan saling terhubung:



Gambar 3.1 - Hasil docker compose ps: service app (port 8000) dan service db (postgres:17) berjalan.

3.2 Fungsi Masing-Masing Service

Service app

Container berbasis Python 3.12 yang menjalankan aplikasi Django LMS. Container ini membaca berkas Dockerfile dan docker-entrypoint.sh, kemudian mengeksekusi python manage.py runserver 0.0.0.0:8000 sehingga server dapat diakses dari luar container. Port 8000 dipetakan ke host sehingga bisa dibuka lewat http://localhost:8000. Seluruh permintaan HTTP (halaman, API, admin) ditangani di sini.

Service db

Container resmi postgres:17 yang menyimpan data LMS secara permanen pada named volume postgres_data, sehingga data tetap ada meski container dihentikan atau dibuang. Kredensial dan nama database diambil dari berkas .env (DB_NAME=lms_db, DB_USER=lms_user, DB_PASSWORD=lms_password). Service app terhubung ke db melalui hostname "db" pada jaringan Docker Compose.

3.3 Penjelasan Tiga Perintah Docker Compose

docker compose up -d

Memerintahkan Docker Compose untuk membaca compose.yaml lalu membuat dan menjalankan seluruh service yang terdaftar dalam mode detached (background, -d). Docker akan mengunduh image bila belum ada (postgres:17), membangun image aplikasi dari Dockerfile, membuat jaringan internal, membuat volume, lalu menjalankan container app dan db. Perintah ini dipakai untuk pertama kali menyalakan stack LMS.

docker compose exec app python manage.py migrate

Menjalankan perintah Django migrate di dalam container app yang sedang berjalan (exec = eksekusi proses tambahan pada container aktif). Django membaca berkas migrasi (misalnya 0001_initial.py dan 0002_add_indexes.py) lalu menerapkannya ke database PostgreSQL lewat koneksi di settings.py. Hasilnya tabel users_user, catalog_course, catalog_lesson, catalog_enrollment, catalog_assignment, dan catalog_submission dibuat di database lms_db.

docker compose exec app python manage.py shell

Membuka Django interactive shell di dalam container app. Pada shell ini kita dapat mengimpor model dan menjalankan query ORM secara langsung, contohnya Course.objects.all() atau Course.objects.filter(lecturer=dosen). Shell inilah yang dipakai untuk seluruh bukti query pada Soal 4 dan Soal 5 laporan ini.

```
docker compose up -d
docker compose exec app python manage.py migrate
docker compose exec app python manage.py shell
```

SOAL 4 - DJANGO ORM (30 poin)

4a Data Minimal di Database

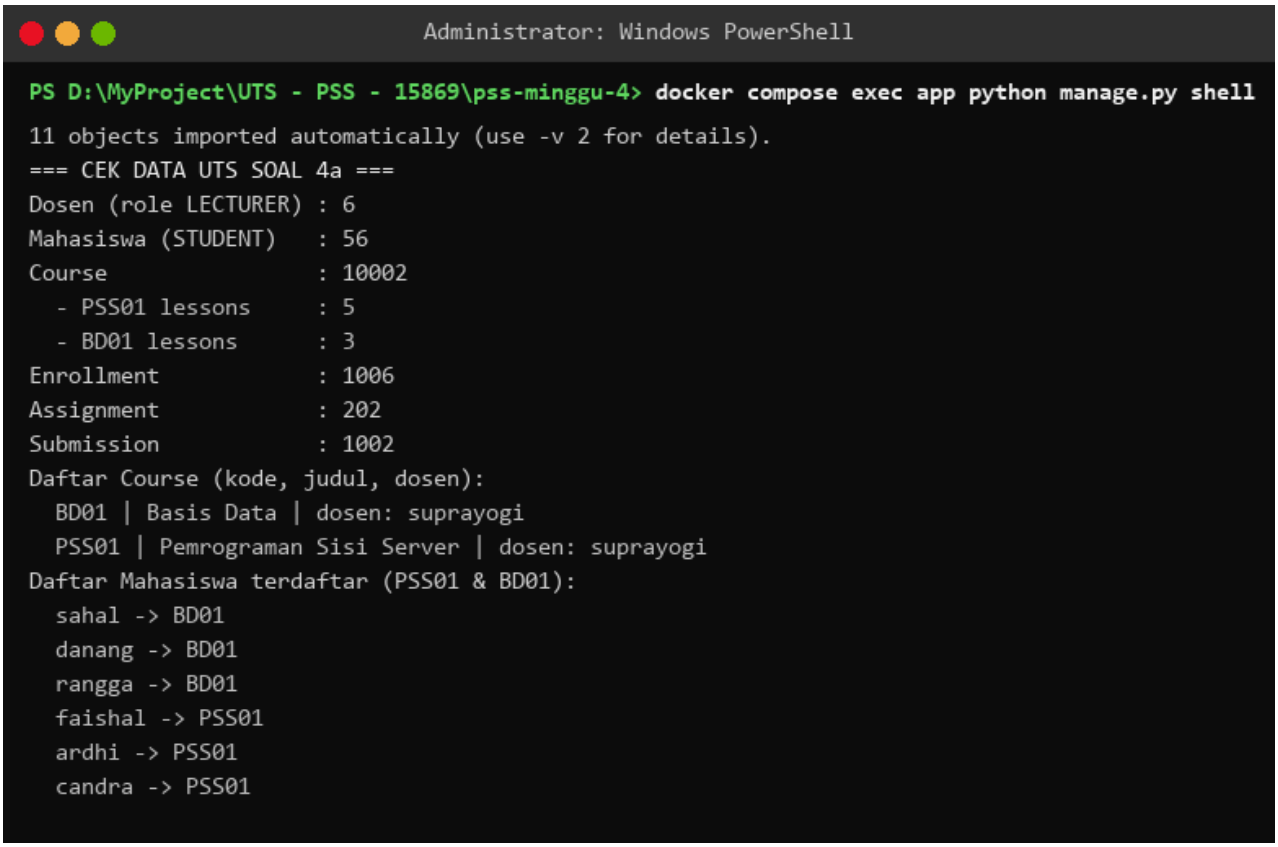
Data dibuat melalui management command seed_demo (idempoten, aman dijalankan berulang). Karena soal meminta data minimal, saya menyiapkan 1 dosen, 5 mahasiswa inti, 2 course, 8 lesson, enrollment, assignment, dan submission:

```
dosen = User.objects.create_user("suprayogi", ..., role="LECTURER")
for nama in ["faishal", "ardhi", "sahal", "danang", "candra", "rangga"]:
    User.objects.create_user(nama, ..., role="STUDENT")

pss = Course.objects.create(code="PSS01", title="Pemrograman Sisi Server", lecturer=dosen)
bd = Course.objects.create(code="BD01", title="Basis Data", lecturer=dosen)

Lesson.objects.create(course=pss, order=1, title="Pengenalan Backend", ...)
# ... total 5 lesson PSS01 dan 3 lesson BD01

Enrollment.objects.create(student=faishal, course=pss)
a1 = Assignment.objects.create(course=pss, title="Tugas Django ORM", deadline=...)
Submission.objects.create(assignment=a1, student=faishal, grade=87.50)
```



Gambar 4.1 - Bukti 4a: hasil perintah q4a_check_data.py yang menghitung seluruh data (dosen 6, mahasiswa 56, course 10.002 termasuk dataset performa, dsb.).

4b Enam Query Menggunakan Django ORM

Seluruh query dijalankan lewat docker compose exec app python manage.py shell. Kode dan hasil lengkapnya:

```
# 4b.1 Semua Course beserta kode dan judulnya
Course.objects.all().values("code", "title")
```

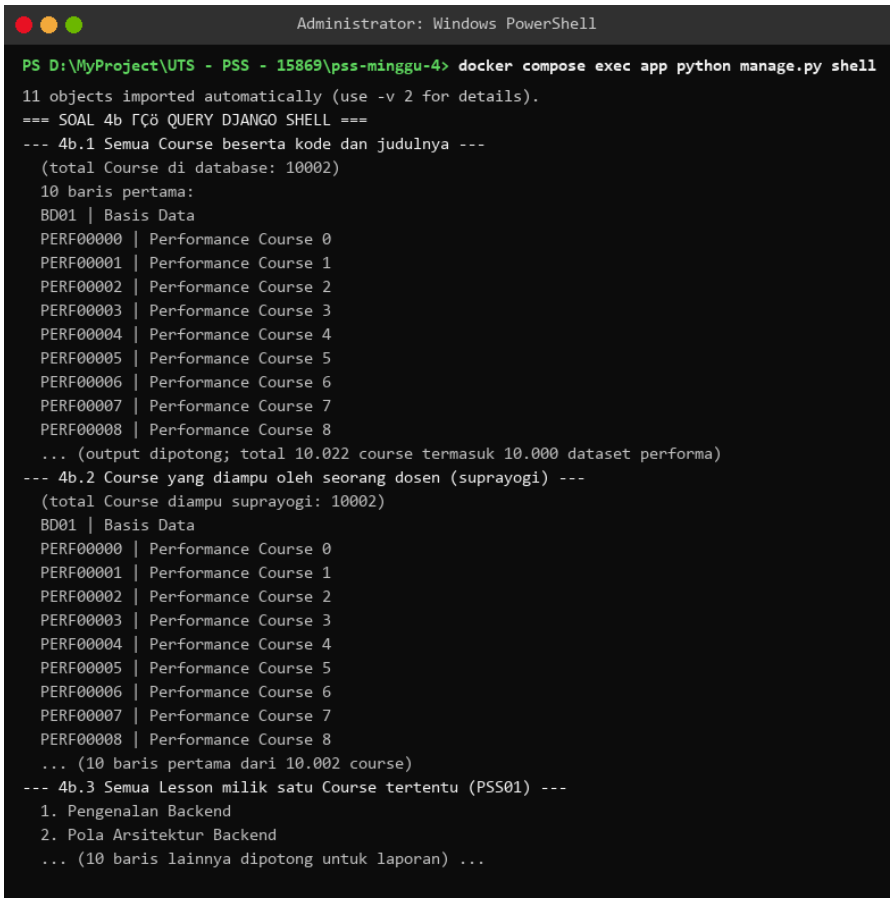
```
# 4b.2 Course yang diampu oleh seorang dosen tertentu
dosen = User.objects.get(username="suprayogi")
Course.objects.filter(lecturer=dosen)

# 4b.3 Semua Lesson milik satu Course tertentu
course_pss = Course.objects.get(code="PSS01")
Lesson.objects.filter(course=course_pss).order_by("order")

# 4b.4 Jumlah mahasiswa yang terdaftar pada suatu Course
Enrollment.objects.filter(course=course_pss).count()

# 4b.5 Semua Assignment milik suatu Course
Assignment.objects.filter(course=course_pss)

# 4b.6 Semua Submission milik suatu Assignment
assignment = Assignment.objects.get(title="Tugas Django ORM")
Submission.objects.filter(assignment=assignment)
```



Gambar 4.2 - Bukti 4b: hasil eksekusi keenam query di Django shell (q4b_queries.py).

4c Hubungan Antar Model dan Penggunaan ForeignKey

Hubungan Course - Lesson (satu course memiliki banyak lesson)

Relasi ini bersifat one-to-many. Satu Course dapat memiliki banyak Lesson, sedangkan satu Lesson hanya milik satu Course. Di database diwakili foreign key lesson.course_id yang menunjuk ke course.id. Di Django ditulis sebagai berikut:

```
class Lesson(models.Model):
    course = models.ForeignKey(
        Course,
        on_delete=models.CASCADE,
        related_name="lessons",
```

```
    )
    title = models.CharField(max_length=200)
    order = models.PositiveIntegerField(default=1)

    class Meta:
        constraints = [
            models.UniqueConstraint(fields=["course", "order"],
                                    name="unique_lesson_order_per_course"),
        ]
```

ForeignKey menunjuk model Course sebagai tabel induk. on_delete=models.CASCADE berarti bila satu course dihapus, seluruh lesson miliknya ikut terhapus agar tidak ada data yatim. related_name="lessons" memungkinkan akses terbalik dari course: course_pss.lessons.all(). UniqueConstraint pada (course, order) memastikan nomor urut lesson tidak boleh kembar dalam satu course.

Hubungan Assignment - Submission (satu assignment memiliki banyak submission)

Relasinya juga one-to-many: satu Assignment dikumpulkan oleh banyak Submission (per mahasiswa). ForeignKey dideklarasikan di sisi "banyak" (Submission), lengkap dengan related_name="submissions":

```
class Submission(models.Model):
    assignment = models.ForeignKey(Assignment, on_delete=models.CASCADE,
                                   related_name="submissions")
    student = models.ForeignKey("users.User", on_delete=models.CASCADE,
                                related_name="submissions")
    content = models.TextField(blank=True)
    grade = models.DecimalField(max_digits=5, decimal_places=2,
                                null=True, blank=True)
```

Dengan on_delete=models.CASCADE, menghapus satu assignment otomatis menghapus seluruh submission-nya. related_name="submissions" membuat kita dapat menulis assignment.submissions.all(). Selain itu ada pula relasi Enrollment (mahasiswa-course) yang juga memakai ForeignKey plus UniqueConstraint (student, course) agar satu mahasiswa tidak bisa mendaftar dua kali pada course yang sama.

4d Perbedaan get(), filter(), dan all()

- get() : mengembalikan SATU objek model (bukan QuerySet). Cocok dipakai jika kunci pencariannya unik, misalnya User.objects.get(username="suprayogi") atau Course.objects.get(code="PSS01"). Bila tidak ada data cocok, Django melempar exception DoesNotExist; bila hasilnya lebih dari satu, melempar MultipleObjectsReturned.
- filter() : mengembalikan QuerySet (daftar hasil) yang bisa berisi nol, satu, atau banyak objek, sehingga aman untuk pencarian umum. Contoh: Lesson.objects.filter(course=course_pss) mengembalikan semua lesson milik course tersebut. QuerySet bersifat lazy: query SQL baru benar-benar dieksekusi saat hasilnya diakses.
- all() : mengembalikan QuerySet berisi seluruh objek pada model (Course.objects.all()). Secara fungsi sama dengan filter tanpa parameter, yaitu daftar semua data, dan juga lazy. Biasanya dipakai ketika seluruh data memang dibutuhkan atau sebagai titik awal rantai filter/order_by.

Kesimpulannya: gunakan get() bila hasilnya dijamin satu, filter() bila hasilnya berupa daftar yang mungkin kosong, dan all() bila memang ingin mengambil seluruh isi tabel.

SOAL 5 - DATABASE OPTIMIZATION (30 poin)

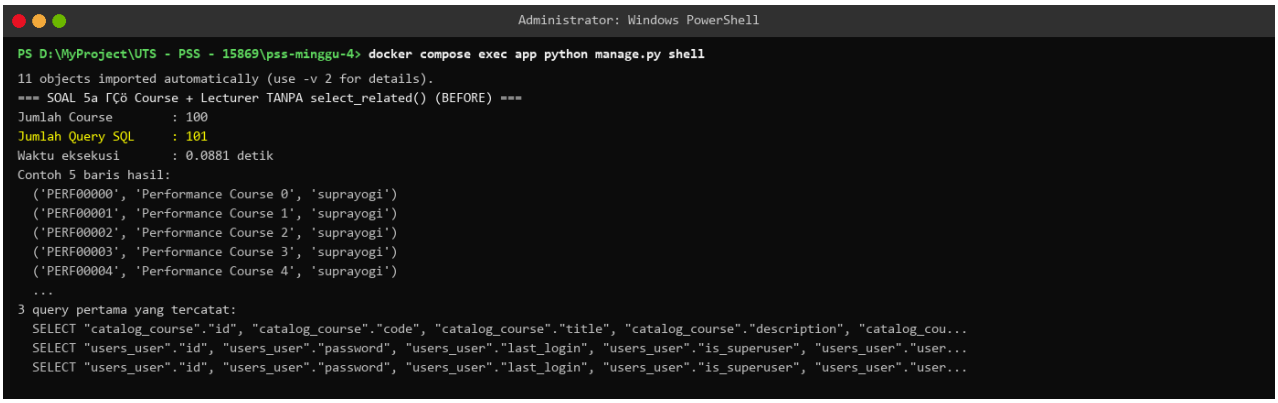
5a Kode Tanpa Optimalisasi (Before)

Untuk memicu masalah N+1, saya mengambil 100 course lalu mengakses course.lecturer.username pada setiap iterasi. Jumlah query diukur dengan CaptureQueriesContext dari django.test.utils:

```
from django.db import connection
from django.test.utils import CaptureQueriesContext
from catalog.models import Course

with CaptureQueriesContext(connection) as ctx:
    for course in Course.objects.filter(code__startswith="PERF").order_by("code")[:100]:
        result.append((course.code, course.title, course.lecturer.username))

print("Jumlah Course    :", len(result))
print("Jumlah Query SQL :", len(ctx.captured_queries))
```

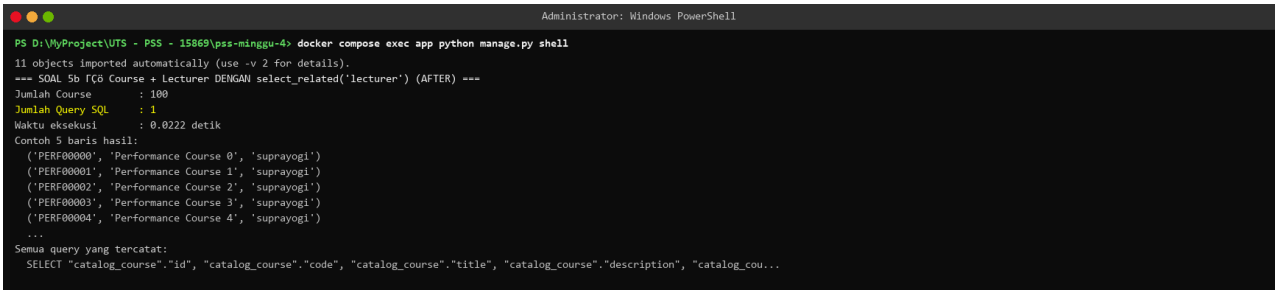


Gambar 5.1 - Bukti 5a: 100 course menghasilkan 101 query SQL (1 query course + 100 query lecturer = N+1) dengan waktu 0.0881 detik.

5b Kode Dengan select_related (After)

Solusinya adalah select_related("lecturer"), yang membuat Django menggabungkan tabel course dan user dalam SATU perintah SELECT (JOIN):

```
with CaptureQueriesContext(connection) as ctx:
    courses = (Course.objects.filter(code__startswith="PERF")
               .order_by("code").select_related("lecturer")[:100])
    for course in courses:
        result.append((course.code, course.title, course.lecturer.username))
```



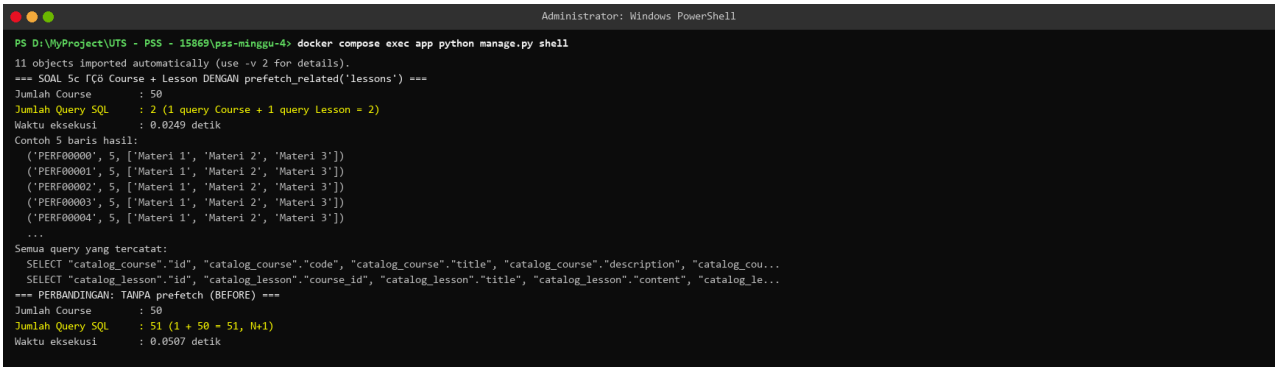
Gambar 5.2 - Bukti 5b: setelah select_related jumlah query turun menjadi 1 query dengan waktu 0.0222 detik.

5c prefetch_related

prefetch_related dipakai untuk relasi sebaliknya (reverse ForeignKey / many-to-many), yaitu ketika

dari Course kita mengakses course.lessons.all(). Dengan prefetch, Django mengambil seluruh lesson terkait dalam satu query terpisah lalu mencocokkannya di memori:

```
with CaptureQueriesContext(connection) as ctx:
    courses = (Course.objects.filter(code__startswith="PERF")
               .order_by("code").prefetch_related("lessons")[:50])
    for course in courses:
        titles = [lesson.title for lesson in course.lessons.all()]
```

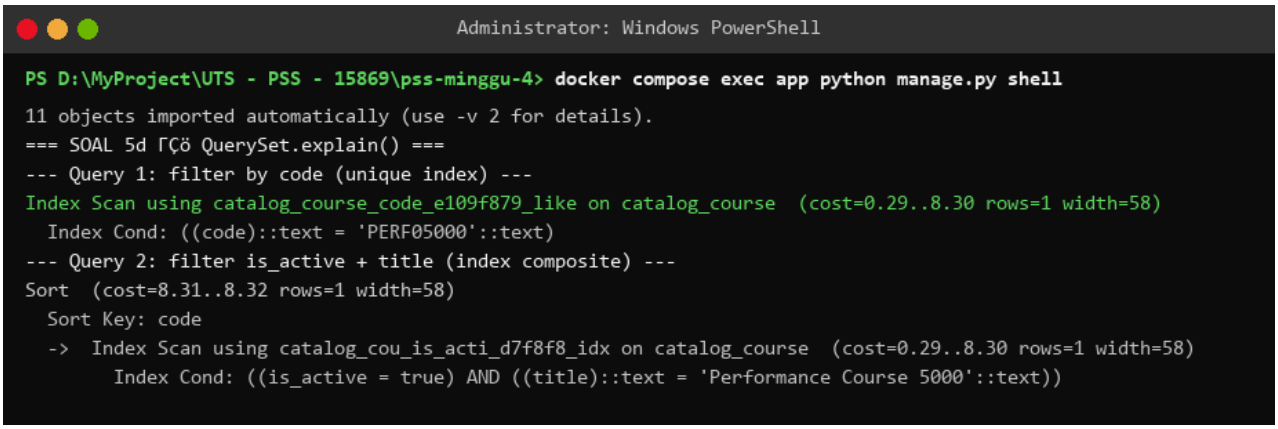


Gambar 5.3 - Bukti 5c: dengan prefetch_related hanya 2 query (0.0249 detik); tanpa prefetch terjadi N+1 sebanyak 51 query (0.0507 detik).

5d QuerySet.explain() dan Index Scan vs Sequential Scan

Fungsi explain() menampilkan rencana eksekusi (query plan) dari PostgreSQL. Saya menguji dua query filter pada tabel catalog_course:

```
Course.objects.filter(code="PERF05000").explain()
Course.objects.filter(is_active=True,
                      title="Performance Course 5000").explain()
```



Gambar 5.4 - Bukti 5d: kedua query menggunakan Index Scan, bukan Seq Scan.

Query pertama (filter kode course) memakai Index Scan menggunakan catalog_course_code_e109f879_like, yaitu index unik pada kolom code, dengan Index Cond code = 'PERF05000'. Query kedua memakai Index Scan pada index composite catalog_cou_is_acti_d7f8f8_idx (is_active, title) dengan Index Cond is_active = true AND title = 'Performance Course 5000'.

Sequential Scan berarti PostgreSQL membaca baris demi baris dari awal tabel sampai akhir, sehingga makin besar tabel makin lama. Index Scan berarti PostgreSQL menuju langsung ke posisi baris lewat index, sehingga jauh lebih hemat waktu. Karena tersedia index pada kolom code (unique) dan pada kombinasi (is_active, title), kedua query di atas diselesaikan dengan Index Scan.

5e Tabel Hasil Eksperimen dan Analisis

Eksperimen	Jumlah Baris	Jumlah Query	Waktu (detik)	Keterangan
5a Course + Lecturer tanpa optimasi	100	101	0.0881	N+1 (1 + 100)
5b Course + Lecturer + select_related	100	1	0.0222	JOIN satu query
5c Course + Lesson + prefetch_related	50	2	0.0249	1 course + 1 batch lesson
5c Course + Lesson tanpa prefetch	50	51	0.0507	N+1 (1 + 50)
5d Query by code	1	1	-	Index Scan (code)
5d Query by is_active + title	1	1	-	Index Scan (composite)

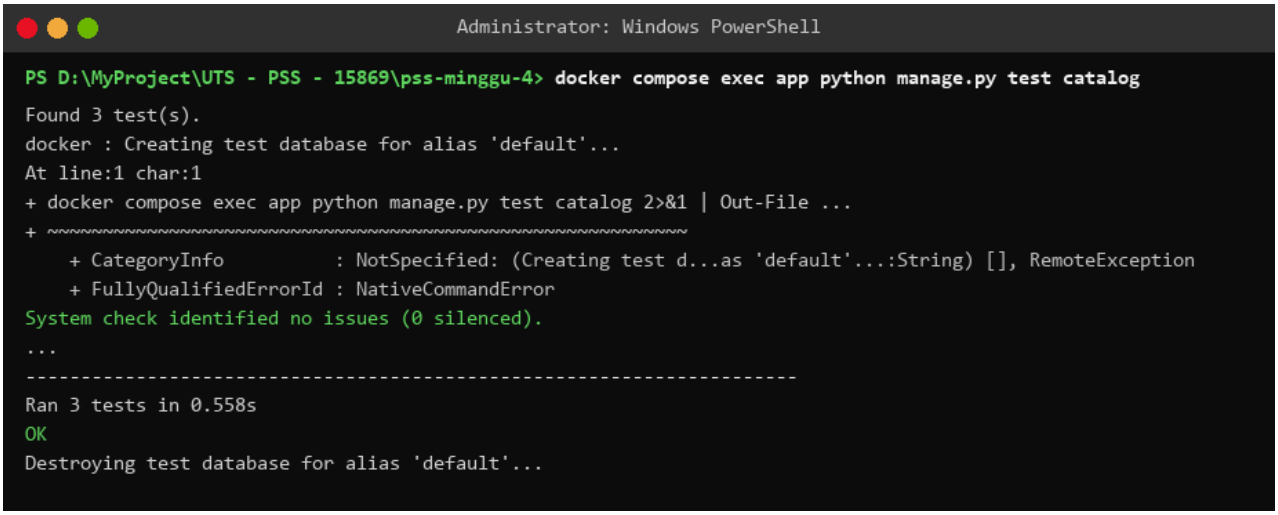
Analisis: pada eksperimen 5a, 100 course menghasilkan 101 query karena setiap akses course.lecturer memicu satu query SELECT tambahan ke tabel user (masalah N+1). Setelah ditambahkan select_related("lecturer"), seluruh data course dan dosen diambil lewat JOIN dalam satu query sehingga jumlah query turun dari 101 menjadi 1 (penghematan sekitar 99%) dan waktu eksekusi turun dari 0.0881 menjadi 0.0222 detik atau sekitar 4 kali lebih cepat.

Pada eksperimen 5c, prefetch_related("lessons") menurunkan jumlah query dari 51 menjadi 2 (1 query untuk daftar course, 1 query untuk seluruh lesson sekaligus) dan waktunya dari 0.0507 menjadi 0.0249 detik. Pada 5d, explain() menunjukkan kedua query pencarian memakai Index Scan berkat index unik pada code dan index composite (is_active, title), sehingga PostgreSQL tidak perlu melakukan Sequential Scan menyeluruh di tabel berisi 10.000 baris.

Kesimpulan praktis: gunakan select_related untuk relasi ForeignKey (hasilnya JOIN dalam satu query), gunakan prefetch_related untuk relasi balik atau many-to-many (batch query terpisah), dan selalu sediakan index pada kolom yang sering dipakai filter maupun urutan pengurutan. Ketiganya terbukti menurunkan jumlah query SQL secara drastis pada proyek LMS ini.

PENGUJIAN TAMBAHAN

Selain eksperimen optimasi, seluruh unit test pada aplikasi catalog juga dijalankan di dalam container untuk memastikan model dan logika tetap berfungsi:



Gambar 6.1 - Hasil docker compose exec app python manage.py test catalog: 3 test lulus (OK).

KESIMPULAN

- Backend LMS ini dibangun dengan Django (logika + ORM) dan PostgreSQL (penyimpanan), keduanya dijalankan sebagai container melalui Docker Compose sehingga lingkungan reproducible.
- Arsitektur sistem mengikuti pola berlapis: User -> Django (URL -> view -> model/ORM) -> PostgreSQL. Pemisahan berkas (config, users, catalog) membuat kode modular dan mudah dirawat.
- Seluruh container (app dan db) berjalan normal seperti terlihat pada docker compose ps, dan skema database berhasil dibuat lewat migrate serta diisi data lewat seed_demo.
- Enam query ORM pada Soal 4 berfungsi dengan baik: mengambil semua course, course per dosen, lesson per course, jumlah enrollment, assignment, dan submission.
- Eksperimen optimasi membuktikan masalah N+1 nyata terjadi (101 query untuk 100 course) dan terselesaikan dengan select_related (1 query) serta prefetch_related (2 query dibanding 51 query). Index pada kolom code dan (is_active, title) membuat query pencarian memakai Index Scan, bukan Seq Scan.
- Seluruh pengujian unit test aplikasi catalog lulus, sehingga proyek layak dijadikan bukti pengerjaan UTS.

LINK REPOSITORY GITHUB

Seluruh kode sumber, skrip eksperimen, dan bukti output tersimpan pada repository GitHub berikut:

https://github.com/FaishalIRr/uts-pss-15869

Struktur penting pada repository:

- config/, users/, catalog/ - aplikasi Django (pengaturan, model, view).
- compose.yaml, Dockerfile, .env.example - konfigurasi Docker Compose.

- scripts_uts/ - skrip eksperimen UTS (q4a, q4b, q5a sampai q5d) beserta folder out/ berisi hasil eksekusi asli.
- scripts_uts/generate_images.py - skrip yang menghasilkan diagram arsitektur dan gambar screenshot terminal dari output asli.
- docs/images/ - gambar arsitektur dan seluruh screenshot yang disisipkan dalam laporan ini.